QuipKeys: Decoding Imprecise Physical-Keyboard Typing with a Context-Aware Noisy-Channel Model

Hasnat

https://github.com/codinghasnat/quipkeys

November 2026

Abstract

Phone keyboards stopped asking for precision years ago: they decode where a finger probably meant to land. Physical keyboards never made that transition, and desktop autocorrect remains a conservative dictionary spellchecker. We present QuipKeys, an offline decoder for physical-keyboard typing that aims to let people type fast and imprecisely, without looking and without reaching for backspace, and still get the words they intended. QuipKeys combines a keyboard-geometry error model, a weighted edit-distance search over a dictionary trie, an interpolated trigram language model built from 1.4 billion words of text, and a sliding-window lattice decoder whose confidence is the posterior probability of each correction summed over all decoding paths. Safety mechanisms (name and code protection, a morphology guard, hysteresis, undo-driven learning) keep false corrections near one in a thousand correctly typed words. Beyond letter slips, it repairs misplaced spaces, whole-hand shifts, Shift-key slips and accidental Caps Lock.

On a held-out benchmark of simulated sloppy typing, QuipKeys fixes 77.3% of mistyped words at moderate error rates (a frequency-only spellchecker: 53.5%) and lifts word accuracy for "blind" typing (10% of letters wrong) from ?% to ?%. In a first real-typing study with one participant typing 33 passages at a median of 85 words per minute with the text hidden and backspace disabled, words typed exactly right rose from 75.4% to 91.5% (92.5% with an error model fitted to the participant), and the held-out half of the data shows the gains generalise. The real data also overturned two assumptions the simulator had baked in: this typist's extra letters are rarely neighbouring keys, so keyboard geometry contributed nothing on real typing, and swapped letters happen at normal speed rather than as fast roll-overs. We release the engine, benchmark and data-collection tools, and discuss what remains before such a decoder could replace autocorrect.

1 Introduction

Typing accurately on a physical keyboard is slow. Skilled typists spend a large share of their keystrokes on correction: noticing an error, backspacing over it, retyping. Large-scale keystroke studies show that corrections are a substantial part of everyday typing, even for fast typists [3]. The cost is not only the keystrokes. Watching for errors pulls attention from what one is writing.

Touchscreen keyboards solved a harder version of this problem. A finger on glass has no tactile boundary between keys, so phone keyboards treat each tap as noisy evidence and decode the intended word with a language model [6, 10, 16]. Users learned they can type sloppily on a phone and mostly get the right words. Physical keyboards never got that treatment. Desktop autocorrect, in the operating system or the word processor, is still a cautious dictionary spellchecker. It fixes `teh`, but it does not reason about which key a finger probably meant, it rarely uses context, and it does not repair misplaced spaces.

QuipKeys asks what happens if a physical keyboard gets a real decoder. Our north star is a typist who does not care where their fingers land. There is still intent, but each key press lands only roughly near the intended key, and the typist never looks down or backspaces. The metric that matters is not spelling accuracy but *effective words per minute*: correct words produced per minute, after all corrections.

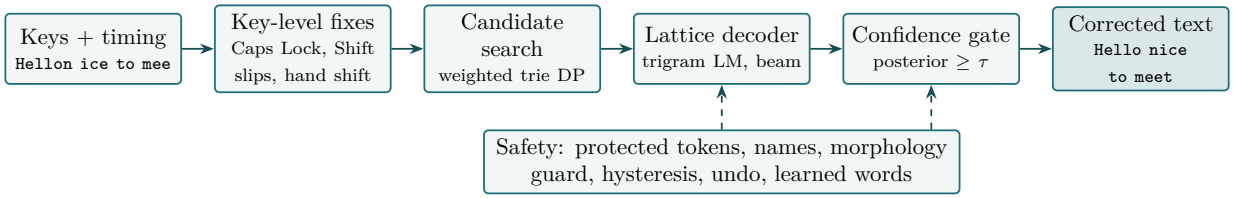


Figure 1: The QuipKeys pipeline. Each chunk is decided at its space; the last four chunks are re-decoded jointly, so a later word can fix an earlier one.

Contributions.

1. A complete offline decoder for physical-keyboard input (§3). It runs in a few milliseconds per word in pure Rust: a geometry- and timing-aware error model, a weighted trie search, an interpolated trigram language model, and a sliding-window lattice decoder with path-summed confidence.
2. A set of safety mechanisms aimed at the failure that makes people turn autocorrect off: changing what they meant to type (§3.5).
3. Handling of error types that word-level spellcheckers do not see: misplaced and missing spaces, whole-hand shifts, Shift-key slips and accidental Caps Lock (§3.6).
4. A benchmark with a simulated sloppy typist at five error levels, including a "rough aim" model in which every press is Gaussian-distributed around the intended key (§4).
5. A first real-typing evaluation: 33 passages from one participant, with error-profile analysis that exposed where the simulator was wrong (§5.2).

2 Related Work

Spelling correction. Edit distance [2, 11] underlies most spelling correction. The noisy-channel formulation [1, 9] chooses the intended word w that maximises $P(t \mid w)\,P(w)$ for typed string t. Real-word errors, typos that happen to produce another valid word, need context, and were addressed early with n-gram models [12]. SymSpell [5] popularised fast candidate generation by symmetric deletion. These systems model cognitive misspellings (`recieve`) at least as much as motor slips, and they correct one word at a time.

Decoding touch input. Soft-keyboard work combined key-press likelihoods with language models from early on [6]. Shape writing decodes whole gestures [10], and VelociTap showed that decoding whole sentences of very imprecise taps supports fast and accurate entry [16]. Our lattice decoder follows the same principle, applied to discrete key events from a physical keyboard, where the only spatial evidence is *which* key fired.

Typing behaviour and data. Large keystroke datasets [3] and corpora of real typos [8] document how people mistype. We could not access these in our build environment, which is one reason our real-typing study collects its own keystroke logs.

3 System

Figure 1 shows the pipeline. Text arrives as a stream of typed chunks separated by spaces, each with per-key timestamps. Every chunk is decided when its space is pressed, so corrections appear immediately. The recent chunks are also re-decoded jointly as more context arrives.

Table 1: Default error-model costs (nats) and the profile fitted to the study participant (§5.2).

Operation	Default	Fitted
Substitution, adjacent key	5.5	5.5
per extra key of distance	+3.0	+2.0
Extra key: repeat / adjacent / other	5.5 / 6.0 / 11.0	5.5 / 6.0 / 7.0
Missing letter: doubled / other	4.0 / 6.5	4.0 / 4.5
Transposition (fast / normal / slow)	3.0 / 5.0 / 6.5	4.5 (no timing)

3.1 Formulation

For a typed chunk t with key times $\mathbf{k}$ and context c, the decoder seeks

$$\hat{w} = \arg\max_{w \in \mathcal{H}(t)} P(t, \mathbf{k} \mid w)\, P(w \mid c), \tag{1}$$

where $\mathcal{H}(t)$ contains dictionary candidates, run-together segmentations (`thecat` $\rightarrow$ *the cat*), session-learned words, and a *keep* hypothesis: the typed string is exactly what was meant. All scores are costs in nats ($-\ln p$). The keep hypothesis is essential. For a string the dictionary does not know, its cost is $\kappa_0 + \kappa_1 \min(|t|, L)$, a prior that the person meant an unknown word (a name, a technical term). Its slope makes long random strings unlikely as intentional words, and the cap L=10 stops long genuine technical words from being "corrected".

3.2 Error model and candidate search

The channel cost of aligning intended letters to typed keys uses a weighted Damerau–Levenshtein recurrence with operation costs drawn from typing behaviour (Table 1):

- a substitution costs more the farther apart the two keys are on a row-staggered QWERTY layout;
- an inserted key is cheap when it repeats or neighbours an adjacent typed key (a fat finger);
- dropping one of a doubled letter, or an apostrophe, is cheap;
- a transposition can be made cheaper when the two keys came within 45 ms (a roll-over).

Errors in the first letter carry a small surcharge.

Candidates are found by running this recurrence directly over a compact trie of about 86,000 words, pruning any branch whose cheapest partial alignment exceeds a bound. The bound grows with word length (2.5 nats per letter beyond five, capped at 19), so long words can absorb more slips. A per-thread cache avoids repeating the search when the same chunk is re-examined at the next space.

3.3 Language model

Context comes from an interpolated trigram model. Unigram frequencies combine the SymSpell lists derived from Google Books [5, 13] with common words from `wordfreq` [15]. We filtered the latter for web misspellings (§5.4). Bigram and trigram counts come from all 1.4 billion words of the English Wiki-40B corpus [7], plus conversational text (fiction dialogue, web chat, film reviews), weighted up because typing is conversational. The final store holds 11.5M word pairs and 13M word triples in 62 MB. Bigram probabilities take the larger of the Google Books and corpus estimates, since each list misses pairs the other has. The trigram weight grows with the count of its two-word context:

$$P(w \mid u, v) = \lambda_3 \frac{c(u, v, w)}{c(u, v)} + (1 - \lambda_3)\big[\lambda P_2(w \mid v) + (1 - \lambda) P_1(w)\big], \tag{2}$$

where $\lambda_3 = 0.7\, c(u, v)/(c(u, v) + 5)$ and $\lambda = 0.6$.

3.4 Sliding-window lattice decoder

Deciding each word alone throws away information. In `i hopw ypur`, neither `hopw` nor `ypur` provides good context for the other until both are read. QuipKeys therefore re-decodes the last four chunks as a lattice. Each chunk offers single readings (keep, the top spelling candidates, splits, learned words). Each adjacent pair offers boundary readings: a merge (`meetin g`) and the space moved one letter either way (`hellon ice`, `mee tyou`). A beam search over states of the last two words [14] finds the best path under the trigram model.

Path-summed confidence. A change at chunk k is applied only if its posterior clears a threshold τ. Crucially, the posterior is the probability mass of *all* paths that produce the same text at k:

$$P(\text{text}_k = y) = \frac{\sum_{\pi:\,\text{text}_k(\pi)=y} e^{-S(\pi)}}{\sum_{\pi} e^{-S(\pi)}}, \tag{3}$$

computed with a beam-pruned forward pass. An early version compared the best path only against the best competing path. It refused `th efriend` → *the friend*, because two different paths (move the space, or drop the stray `e`) produce the same text, and each made the other look like a rival. Summing over paths fixed this. The window is reset at sentence boundaries. The thresholds are τ=0.9 for non-words and τ=0.98 for changing a real word, which also carries an extra cost of 1.5 nats.

3.5 Safety mechanisms

A decoder that silently changes correct text is worse than none. QuipKeys uses several layers of protection:

- **Protected tokens.** Anything code-like is never touched: digits, `snake_case`, `camelCase`, paths, URLs, e-mail addresses, flags (`-force`), ALL-CAPS acronyms, dialect elisions (`countin'`) and abbreviations (`Mr.`).
- **Names.** A capitalised word mid-sentence that is not in the dictionary is treated as a name and needs 99.9% confidence. It can be fixed toward a known proper noun only if the slip is a single clean keyboard slip (adjacent key, roll-over, doubled or dropped repeat) and never in the first letter. This rule separates `Calicornia` → *California* from `Colman` → *Coleman*: names differ from one another by spelling-variant patterns, typos by keyboard patterns. Words normally capitalised (learned from Wikipedia casing) are restored (`london` → *London*).
- **Morphology guard.** A string that is a known stem plus a regular affix (`freewheelers`, `banquetings`) is scored like a real word.
- **Hysteresis.** Each word changes at most once. The single exception is a boundary repair, which may override an earlier spelling fix of the word before it.
- **Undo and learning.** Undoing a correction restores the typed word, protects it, and adds it to the session's learned words. A word typed identically twice without correction is learned too. It is protected from then on, and later typos of it are fixed toward it (`kubetcl` → *kubectl*).
- **English variety.** When a typo is corrected and both UK and US spellings exist (1,115 pairs), the preferred variety (UK by default) is favoured. Correctly typed words of the other variety are never changed.

Table 2: Simulated typing levels.

Level	Letters mistyped	Words mistyped	Notes
light	1.5%	≈7%	careful typist
moderate	3%	≈14%	
messy	6%	≈26%	
blind	10%	≈37%	not looking at the keys
rough	Gaussian aim	≈49%	fingers land roughly near the key

3.6 Errors beyond letters

- **Spacing.** Missing spaces are handled by segmentation, with exact pieces or one fuzzy piece. Premature and shifted spaces are handled by the lattice's boundary readings. A merge of two real words needs 6 extra nats of evidence, because merges are how compounds get mangled (`soy beans`).
- **Hand shift.** A word typed with the whole hand one key over (`jr;;p` → *hello*) is detected by remapping it.
- **Shift-key slips.** Patterns such as `THe`, `hTe` and `uSre` are recognised. A capital in second position signals that the first two keys rolled over (*Sure*).
- **Accidental Caps Lock.** Three inverted-case words in a row (`w NEED TO`) switch on case flipping. The flip is applied to those words and to later mostly-uppercase words, which also covers mistimed Shift under Caps Lock (`thANSK` → *Thanks*). Real acronyms (`NASA`) do not trigger it.

3.7 Adapting to sloppiness

Rough aim calls for a different error model: adjacent slips become the norm and distant keys stay unlikely. QuipKeys keeps two models and estimates sloppiness online as the share of the last 50 words that were typed as non-words. It switches to the rough-aim model above 45% and back below 35%.

4 Evaluation Methodology

Text. Clean evaluation sentences come from the Brown corpus [4]. It is independent of every data source the engine uses: 1,500 development sentences for all tuning, 1,500 held-out test sentences for reporting, and 60 whole documents for measuring document memory.

Simulated typist. Because no real keystroke data was available at first, a simulated typist types the clean sentences. Its discrete-slip model mixes adjacent-key substitutions, transpositions (with very short inter-key intervals, though short intervals also occur on 6% of correct pairs), omissions, fat-finger double presses and double taps. It also has missing, premature and shifted spaces, and occasional whole-hand shifts. The *rough-aim* model, our north star, lands every press at a Gaussian offset from the intended key's centre ($\sigma = 0.30$ key widths horizontally, 0.24 vertically) and fires the nearest key, with roll-overs and dropped keys on top. Table 2 lists the levels.

Metrics. For mistyped words we report the share *fixed* (output equals intent), the share changed to the *wrong* word, and the share *left* as typed. For correctly typed words we report the *false-correction* rate, the number that decides whether people keep a corrector switched on. Word accuracy is the share of all words output exactly right. An effective-WPM estimate models an

Table 3: Held-out results on simulated typing (1,500 Brown test sentences per level).

Level	System	Fixed	Wrong	False corr.	Word acc.	p50 / p99 (ms)
light	no correction	–	–	–	92.72%	–
	plain	55.0%	2.5%	0.095%	96.64%	0.3 / 2.8
	QuipKeys	78.7%	2.0%	0.135%	98.33%	0.5 / 3.1
moderate	no correction	–	–	–	85.92%	–
	plain	53.5%	3.3%	0.078%	93.39%	0.4 / 3.1
	QuipKeys	77.3%	2.4%	0.132%	96.69%	0.6 / 4.0
messy	no correction	–	–	–	73.98%	–
	plain	51.3%	3.9%	0.080%	87.27%	0.6 / 3.8
	QuipKeys	75.5%	3.3%	0.109%	93.55%	0.8 / 5.3

Table 4: Ablations: share of mistyped words fixed (held-out simulated typing).

Configuration	moderate	messy	blind	rough
QuipKeys (full)	77.3%	75.5%	?%	?%
word-by-word instead of lattice	77.1%	?%	?%	?%
no keyboard geometry	72.3%	70.5%	?%	?%
pairs only (no trigrams)	74.0%	72.8%	?%	?%
no boundary repair	74.4%	72.2%	?%	?%
never switch to rough-aim model	77.3%	?%	?%	?%
plain (frequency-only)	53.5%	51.3%	?%	?%

80 WPM typist who notices and retypes every wrong word; it is a model, not a user measurement. Latency is per word, single-threaded, on a cloud CPU.

Baseline. *plain* is a classic frequency-only spellchecker built from the same components, with no keyboard geometry, no timing, no context, no splits and no lattice, behind the same confidence gate. We could not run the operating systems' own autocorrect in our environment (§6).

5 Results

5.1 Simulated typing

Table 3 gives held-out results. QuipKeys fixes 20–28 points more mistyped words than the frequency-only baseline at every level. Its false-correction rate is higher than the far more passive baseline's but stays near or below one in 700 correctly typed words. For blind typing, word accuracy rises from ?% to ?%.

Ablations. Table 4 switches components off one at a time. On simulated typing, keyboard geometry is the most valuable single component (4–7 points), followed by the trigram context (2–3 points), boundary repair (about 3 points) and the lattice decoder (about 1 point, plus fewer false corrections). The rough-aim error model is worth about 5 points on rough typing, and the switching rule keeps it off for careful typing.

5.2 Real typing

Protocol. One participant, the author, typed 33 thirty-word passages of everyday sentences in a browser-based Typing Lab. The page logged every keystroke with millisecond timing and applied no correction. Runs used the north-star condition: as fast as possible, not looking at the keys, backspace disabled. Most runs also hid the typed text, because seeing it prompted the participant

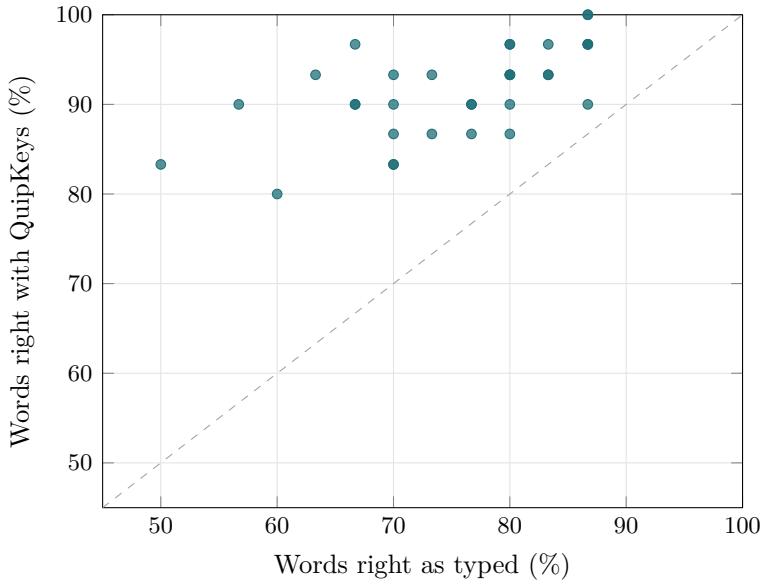


Figure 2: Real typing, one point per 30-word run (33 runs, 75–96 WPM, text hidden, no backspace). Every run lies above the diagonal.

Table 5: Real typing (33 runs, 990 words). "Held-out" is the 21 runs typed after the personal error model was fitted on the first 12.

System	All 33 runs	Held-out 21 runs
As typed	75.4%	75.7%
plain (frequency-only spellchecker)	84.7%	–
QuipKeys, no context	87.6%	–
QuipKeys, no boundary repair	89.9%	–
QuipKeys, pairs only (no trigrams)	90.3%	–
QuipKeys, word-by-word (no lattice)	91.2%	–
QuipKeys, no keyboard geometry	91.6%	–
QuipKeys	**91.5%**	**92.4%**
QuipKeys + fitted error model	**92.5%**	**93.0%**

to go back and fix it. Five early runs were typed with Caps Lock on by accident. The logs were replayed through the engine offline, and outputs were scored against the target words, ignoring case and punctuation, because the participant mostly omitted punctuation.

Accuracy. Table 5 and Figure 2 show the results. The share of words typed exactly right rose from 75.4% to 91.5%. Every run improved, and three reached 96–100%. At the participant's median raw speed of 85 WPM, that is roughly 64 correct words per minute uncorrected versus 78 with QuipKeys. Caps Lock recovery, added after the first 12 runs exposed the problem, lifted those 12 runs (five of them typed with Caps Lock on) from 83.3% to 90.0%.

The real error profile. Table 6 compares the participant's slips with the simulator's assumptions. Two assumptions were wrong. First, most extra letters were *not* neighbours of the keys around them (66 of 100 extra letters). Second, swapped pairs happened at the participant's normal pace (median inter-key interval of about 100–150 ms) rather than as sub-45 ms roll-overs. Consistent with this, keyboard geometry, the most valuable component on simulated data, contributed nothing measurable on real data (Table 5). Context and boundary repair carried the gains. An error model refitted to these frequencies (Table 1) gained 1.7 points on the 12 runs it was fitted to and 0.6 points on the 21 runs typed afterwards.

Table 6: Error profile of real typing (33 runs, 3,944 letters, 346 slips) versus the discrete-slip simulator.

Slip	Real typist	Simulator
Wrong key	32% (71% adjacent)	40% (90% adjacent)
Missing letter	22%	19%
Extra letter, not near neighbours	19%	0%
Extra letter, adjacent or repeat	10%	19%
Swapped pair	14%	22%
Word boundary	4%	≈10%
Letters mistyped	8.8%	10% (blind)

What remains. Of the 81 words still wrong across all runs, half are real-word slips (`ship` for *shop*, `our`/`out`, `nice` for *nine*, `quite`/`quiet`). Trigram context cannot overrule these safely. The other half are non-words with two or more slips or a genuinely ambiguous reading (`asn`: *and* or *an*? `thei`: *this*, *their* or *the*?).

5.3 Speed

Median latency is 0.5–1.0 ms per word, and the 99th percentile is 3–5 ms (8 ms when the rough-aim model is active). That is well below perceptible delay. The engine loads in about 0.3 s and runs unchanged in a browser through WebAssembly.

5.4 Negative results and lessons

- **4-grams.** Adding 6.6M 4-grams from the same 1.4B words gave no gain and more false corrections. They were dropped.
- **A learned decision gate.** A logistic model over 18 decision features, trained with the evaluation's cost weights, improved its training objective. On the benchmark it raised false corrections enough to cancel the gain; restricted to non-word decisions it was roughly neutral, so it was not adopted.
- **A tiny neural language model.** A 4.6M-parameter transformer trained for an hour on CPU (perplexity about 200) more than doubled real-word fixes but cost 4–6 points overall and was 5× slower. A GPU-trained model remains future work; the training pipeline and an int8 Rust inference engine are in place.
- **A document word cache** reinforced uncorrected slips and was replaced by learning only words typed identically twice.
- **Dictionary hygiene matters.** A rule meant to admit UK/US variant pairs (`enrolment` and `enrollment`) also admitted 364 web misspellings (`actualy`, `realy`, `allready`), which the engine then treated as words. Real variants are within about 10× of each other in frequency; misspellings are 300–3000× rarer than the word they misspell. A frequency-ratio filter separates them cleanly. The participant's own typing (`actyaly`) exposed this bug, which the simulator never could.

6 Limitations

- **One participant.** The real-typing study has a single participant, who is also the author. The held-out split guards against fitting to particular runs, not to a particular typist.

- **No comparison with operating-system autocorrect.** We could not run macOS or Windows autocorrect in our environment, so the baseline is our own frequency-only spellchecker.
- **Effective speed versus careful typing is not yet measured.** The key behavioural question is whether fast, sloppy typing with QuipKeys beats careful typing with backspace. The participant's careful baseline was not recorded.
- **A simulator mismatch.** Simulated results are tuned to a simulator that real data showed to be partly wrong (Table 6).
- **Variety mismatch in evaluation.** The evaluation text is 1960s American prose, while the engine prefers UK spellings when correcting.
- **No operating-system integration.** The system has not yet been integrated as an input method, where issues such as secure fields and application compatibility arise.

7 Conclusion and Future Work

QuipKeys shows that the decoding approach that freed phone typists from precision transfers to physical keyboards. On real sloppy typing at 85 WPM, it raised exact-word accuracy from 75% to over 91%, while rarely touching correct text. Real data was decisive. It corrected the simulator, found a dictionary bug, and showed that context, not geometry, carries the gains for this typist.

The next steps are:

- measuring careful-typing and operating-system-autocorrect baselines on the same participant;
- recruiting more typists, including people with motor impairments, for whom imprecision is not a choice;
- a GPU-trained neural re-ranker for real-word slips;
- learning each user's error profile online;
- input-method integration on Windows and macOS.

Acknowledgements. The engine, benchmark and this paper were developed with the help of Claude, an AI assistant by Anthropic.

References

[1] Eric Brill and Robert C. Moore. An improved error model for noisy channel spelling correction. In *Proceedings of the 38th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 286–293, 2000.

[2] Fred J. Damerau. A technique for computer detection and correction of spelling errors. *Communications of the ACM*, 7(3):171–176, 1964.

[3] Vivek Dhakal, Anna Maria Feit, Per Ola Kristensson, and Antti Oulasvirta. Observations on typing from 136 million keystrokes. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*, 2018.

[4] W. Nelson Francis and Henry Kučera. *Brown Corpus Manual*. Department of Linguistics, Brown University, 1979.

[5] Wolf Garbe. SymSpell: Symmetric delete spelling correction algorithm, 2012. Software, https://github.com/wolfgarbe/SymSpell.

[6] Joshua Goodman, Gina Venolia, Keith Steury, and Chauncey Parker. Language modeling for soft keyboards. In *Proceedings of the 7th International Conference on Intelligent User Interfaces (IUI)*, pages 194–195, 2002.

[7] Mandy Guo, Zihang Dai, Denny Vrandečić, and Rami Al-Rfou. Wiki-40B: Multilingual language model dataset. In *Proceedings of the 12th Language Resources and Evaluation Conference (LREC)*, 2020.

[8] Masato Hagiwara and Masato Mita. GitHub typo corpus: A large-scale multilingual dataset of misspellings and grammatical errors. In *Proceedings of the 12th Language Resources and Evaluation Conference (LREC)*, 2020.

[9] Mark D. Kernighan, Kenneth W. Church, and William A. Gale. A spelling correction program based on a noisy channel model. In *Proceedings of the 13th International Conference on Computational Linguistics (COLING)*, 1990.

[10] Per Ola Kristensson and Shumin Zhai. SHARK^2: A large vocabulary shorthand writing system for pen-based computers. In *Proceedings of the 17th Annual ACM Symposium on User Interface Software and Technology (UIST)*, pages 43–52, 2004.

[11] Vladimir I. Levenshtein. Binary codes capable of correcting deletions, insertions, and reversals. *Soviet Physics Doklady*, 10(8):707–710, 1966.

[12] Eric Mays, Fred J. Damerau, and Robert L. Mercer. Context based spelling correction. *Information Processing & Management*, 27(5):517–522, 1991.

[13] Jean-Baptiste Michel et al. Quantitative analysis of culture using millions of digitized books. *Science*, 331(6014):176–182, 2011.

[14] Lawrence R. Rabiner. A tutorial on hidden Markov models and selected applications in speech recognition. *Proceedings of the IEEE*, 77(2):257–286, 1989.

[15] Robyn Speer, Joshua Chin, Andrew Lin, Sara Jewett, and Lance Nathan. LuminosoInsight/wordfreq: v2.2, 2018. Zenodo, doi:10.5281/zenodo.1443582.

[16] Keith Vertanen, Haythem Memmi, Justin Emge, Shyam Reyal, and Per Ola Kristensson. VelociTap: Investigating fast mobile text entry using sentence-based decoding of touchscreen keyboard input. In *Proceedings of the 33rd Annual ACM Conference on Human Factors in Computing Systems (CHI)*, pages 659–668, 2015.